

Laboratorio #5

Realice este laboratorio de forma individual. Suba las respuestas al enlace de Mediación.

Selección Única (20%)

Marque con X la opción correcta. Cada respuesta vale 2%.

- Una plataforma de aprendizaje en línea quiere permitir que los estudiantes trabajen en grupos donde cada grupo contiene estudiantes, tareas y subgrupos. ¿Qué patrón usaría para manejar esta estructura jerárquica?
a) Decorator - permite agregar comportamientos adicionales a un objeto dinámicamente, sin cambiar clase original.
☒ b) Composite - permite tratar objetos individuales y colecciones de la misma forma (ideal para estructuras jerárquicas).
c) Bridge - separa una abstracción de su implementación → varían independientemente (ófil combinación variantes).
d) Mediator - objeto intermediario, controla la comunicación entre múltiples objetos. evitando conexiones directas.
- Un sistema de mensajería interna debe permitir cambiar la forma de envío de notificaciones (correo, SMS, push) sin modificar el código principal del sistema. ¿Qué patrón aplicaría?
a) Adapter permite usar una clase con interfaz incompatible adaptándolo a la interfaz que el cliente espera.
b) Chain of Responsibility permite varios objetos manejen una solicitud, cada uno hasta que alguno lo procesa.
☒ c) Strategy diferentes estrategias de algoritmos intercambiables.
d) Proxy un representante controla el acceso a otro objeto controla permisos.
- En un entorno de trabajo colaborativo, múltiples usuarios editan el mismo documento. El sistema necesita permitir "deshacer" y "rehacer" los cambios. ¿Qué patrón de diseño aplica mejor?
☒ a) Command permite encapsular una acción como un objeto
b) Memento guarda estado interno objeto para restaurarlo luego.
c) Observer notifica automáticamente a múltiples objetos cuando otro objeto cambia.
d) State Cambia el comportamiento de un objeto cuando su estado interno cambia.
- En una arquitectura basada en eventos, ¿qué atributo de calidad se mejora al desacoplar emisores y receptores de mensajes?
a) Seguridad
☒ b) Mantenibilidad
c) Disponibilidad
d) Usabilidad
- El patrón Flyweight mejora principalmente:
a) Usabilidad
b) Escalabilidad horizontal
☒ c) Eficiencia en el uso de memoria
d) Portabilidad del sistema
- El siguiente código rompe un principio SOLID. ¿Cuál?

```

class ReportGenerator {
    public void generateReport(String data) {
        System.out.println("Generando reporte: " + data);
    }
    public void sendEmail(String data) {
        System.out.println("Enviando por correo: " + data);
    }
}

```

- ☒ a) SRP
- b) LSP
- c) DIP
- d) OCP

7. El siguiente código es un ejemplo de mal diseño, ¿por qué?

```

class PaymentProcessor:
    def process(self, card_type):
        if card_type == "VISA":
            print("Procesando VISA")
        elif card_type == "MasterCard":
            print("Procesando MasterCard")
        else:
            print("Tipo no soportado")

```

- ☒ a) Rompe el Principio de OCP
- b) Rompe el Principio de SRP
- c) Afecta la eficiencia del sistema
- d) Afecta la seguridad del sistema

8. ¿Qué atributo de calidad se vería afectado negativamente si un sistema tiene muchas dependencias circulares?

- ☒ a) Mantenibilidad
- b) Rendimiento
- c) Seguridad
- d) Escalabilidad

9. El siguiente código rompe la Inversión de Dependencias (DIP) porque:

```

class EmailService {
    public void send(String msg) {
        System.out.println("Email enviado");
    }
}

class Notification {
    private EmailService email = new EmailService();
    public void notify(String msg) { email.send(msg);}
}

```

- a) Notification depende de una abstracción
- b) EmailService implementa una interfaz
- ☒ c) Notification depende de una clase concreta
- d) EmailService es un Singleton

10. ¿Cuál es el principal tradeoff al aumentar la disponibilidad mediante redundancia?

- a) Reduce la cohesión del sistema
- b) Aumenta la latencia
- ☒ c) Aumenta costos de mantenimiento
- d) Reduce la interoperabilidad

Desarrollo (40%)

1. ¿Qué significa que un sistema tenga alta cohesión y bajo acoplamiento? (5%)

- Alta cohesión: significa que un componente solo tiene una sola responsabilidad, lo que ayuda a tener un solo propósito, mejor rendimiento y menos errores.
- Bajo acoplamiento: significa que los componentes están conectados libremente y pueden funcionar de manera independiente, lo que hace un sistema simple, flexible y robusto.

2. ¿La siguiente historia cumple con INVEST? Justifique cada letra de INVEST. (5%) Como

estudiante, quiero acceder a mis notas para poder verificar mi progreso en los cursos.

SI,

- I: La historia parece independiente, no depende de otra funcionalidad.
- N: La idea principal es clara, pero no totalmente detallada, lo que puede permitir una negociación.
- V: La tarea es valiosa para los estudiantes.
- E: Se puede estimar el esfuerzo para desarrollar esta tarea, sin mucho riesgo.
- S: Es suficientemente pequeña para realizarse en 1 sprint.
- T: Se puede testear que cada estudiante pueda ver su nota.

3. En UML, explique la diferencia entre dependencia y asociación. Dibuje un ejemplo para cada relación. (10%)

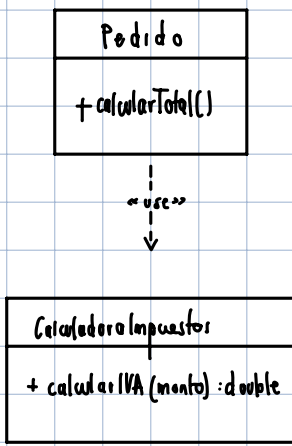
Para las preguntas 4 y 5, por favor lea el siguiente enunciado

Una empresa de transporte público desea desarrollar un sistema para calcular rutas y tarifas. El sistema debe cumplir con los siguientes requisitos:

- Existen distintos medios de transporte (bus, tren, ferry).
- Cada medio calcula su tarifa y tiempo estimado de forma diferente.
- El sistema debe permitir añadir nuevos medios de transporte sin afectar las clases existentes.
- Los cálculos deben poder cambiarse dinámicamente (por ejemplo, según hora pico o clima).
- Existen reglas de negocio dinámicas que pueden otorgar distintos tipos de descuentos o tarifas a pagar. Por ejemplo: descuentos por estudiante, tarifas extra por hora pico, por circular cuando la placa cuenta con restricción, descuentos verdes (por transporte eléctrico) y convenios especiales con municipalidades.
- A futuro, se quiere integrar un algoritmo de optimización de rutas basado en IA que pueda inyectarse sin afectar la arquitectura actual.

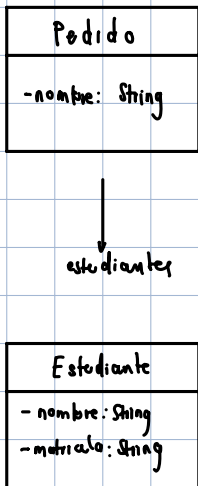
3.

Dependencia



La clase Pedido utiliza temporalmente a CalculadoraImpuestos en su método calcularTotal(), pero no mantiene una referencia permanente.

Asociación



La universidad tiene una asociación estructural con múltiples Estudiantes. Esta relación es permanente y representa una conexión estructural.

Con base en lo anterior, responda:

4. ¿Qué patrón de diseño usaría para representar los diferentes medios de transporte y sus cálculos? (10%)

Patrón: Strategy

Explicación: Permite definir varios algoritmos, en este caso los calcular de tarifa y tiempo estimado para cada medio de transporte, encapsularlos e intercambiarlos.

- Permitiría añadir nuevos transportes sin modificar el código base.

5. ¿Qué patrón de diseño usaría para permitir la integración de algún servicio de IA posteriormente? (10%)

Patrón: Bridge

Explicación: Separa la abstracción (cálculo de rutas) de la implementación (algoritmo IA), así permitiendo que varíen independientemente.

- Se definiría una abstracción para el servicio de optimización de rutas y una implementación concreta del algoritmo IA, sin afectar la arquitectura actual.

Código (40%)

Esta sección continúa con el enunciado dado en el punto 4 y 5 del Desarrollo.

Realice un POC (proof of concept) en código para explicar cómo implementaría los posibles distintos descuentos y tarifas que cambian dinámicamente.

Un ejemplo sobre como puede verse el resultado de su código es:

Calculando viaje combinado: Bus + Tren + Bicicleta

Tarifa base total: 2450 colones

Aplicando descuento de estudiante (-15%)

Aplicando descuento verde (-5%)

Tarifa final: 1970 colones